LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE (PTI23043) • SEMESTER V

Praktikum 04: UX, RecyclerView, Fragment, dan Navigasi

Studi Kasus Aplikasi WargaHub P04 (`id.antasari.p04.wargahub`)

Nama Mahasiswa / NIM	Riyo Wijaya / 240104040160
Mata Kuliah & SKS	Praktikum Pemrograman Mobile (PTI23043) — 3 SKS
Dosen Pengampu	Muhayat, M.IT
Lingkungan & Perangkat Uji	Android Studio, Kotlin, XML Views, ViewBinding, Emulator Medium Phone API 37 (`minSdk 26` , `compileSdk 37`)

1. Sketsa Alur Tiga Layar dan Navigasi

Aplikasi **WargaHub P04** dirancang dengan pendekatan *Single-Activity Architecture* menggunakan satu wadah utama (`MainActivity`) yang memuat `Toolbar` serta `FragmentContainerView` (`NavHostFragment`). Ketiga layar aplikasi direpresentasikan sebagai *Fragment* yang dikelola melalui peta navigasi `res/navigation/nav_graph.xml` .

Layar (Fragment)	Komponen UI Utama	Aksi Navigasi Maju	Aksi Navigasi Kembali
1. Beranda (`HomeFragment`) `fragment_home.xml`	Judul aplikasi (`WargaHub`), pengantar alur baca, catatan batas data contoh, dan tombol aksi utama `btnLihatLaporan` .	Menekan tombol **Lihat Daftar Laporan** menjalankan `action_home_to_list` menuju `ListFragment` setelah memeriksa `currentDestination` .	Merupakan *startDestination*. Menekan tombol *Back* sistem akan menutup atau keluar dari aplikasi secara wajar.
2. Daftar Laporan (`ListFragment`) `fragment_list.xml`	Petunjuk pilih baris, indikator jumlah data (`tvJumlah`), tombol `btnModeKosong` , serta `FrameLayout` berisi `rvLaporan` dan `emptyContainer` .	Memilih satu baris `RecyclerView` menjalankan `action_list_to_detail` sambil mengirim `Bundle` berisi `laporan_id` bertipe `Long` .	Menekan ikon panah *Up* pada `Toolbar` atau tombol *Back* sistem mengembalikan tampilan ke `HomeFragment` .
3. Rincian Laporan (`DetailFragment`) `fragment_detail.xml`	Petunjuk periksa rincian, judul (`tvJudul`), kategori (`tvKategori`), status contoh (`tvStatus`), deskripsi lengkap, dan tombol `btnKembaliDaftar` .	Layar akhir alur pemeriksaan rincian laporan (tidak ada aksi navigasi maju ke layar lain).	Menekan **Kembali ke Daftar** memanggil `navigateUp()` (sama seperti *Up* Toolbar & *Back* sistem) untuk kembali ke daftar yang sama tanpa membuat *instance* baru.

> **Batas Data Simulasi pada Proyek P04:** Seluruh data yang ditampilkan berjumlah 15 laporan lingkungan fiktif yang dibentuk secara lokal di memori melalui objek `LaporanContoh` dengan status `Simulasi`. Fitur tombol *Uji Daftar Kosong* hanya mengganti daftar yang dikirim ke `ListAdapter` menjadi `emptyList()` tanpa menghapus sumber data. Proyek ini belum menerapkan penyimpanan database permanen maupun komunikasi jaringan (API).

2. Penjelasan Konsep Arsitektur: Adapter, Fragment, dan Navigasi

2.1. RecyclerView, ListAdapter, ViewHolder, dan DiffUtil

Untuk menampilkan kumpulan 15 laporan secara efisien, `RecyclerView` menggunakan `LinearLayoutManager` guna menyusun baris secara vertikal dan mendaur ulang komponen tampilan baris (`item_laporan.xml`) yang keluar dari area layar. Kelas `LaporanAdapter` mewarisi `ListAdapter<Laporan, LaporanHolder>` yang memanfaatkan objek `DiffUtil.ItemCallback` bernama `Pembanding`. Fungsi `areItemsTheSame` membandingkan identitas unik laporan (`a.id == b.id`), sedangkan `areContentsTheSame` memeriksa kesamaan seluruh isi properti *data class* (`a == b`).

Di dalam `LaporanHolder`, fungsi `bind()` mengikat objek `Laporan` ke tampilan melalui `ItemLaporanBinding`. Demi mendukung standar aksesibilitas pembaca layar (TalkBack), ketiga `TextView` anak di dalam `item_laporan.xml` diberi atribut `importantForAccessibility="no"`, sementara elemen akar (*root*) baris memiliki tinggi minimum `72dp` dan diisi `contentDescription` gabungan (judul, kategori, status contoh, serta petunjuk aksi *Buka rincian*) agar seluruh baris dibaca sebagai satu kesatuan target sentuh.

2.2. Siklus Hidup Fragment dan Pengelolaan ViewBinding

Berbeda dengan Activity, objek `Fragment` memiliki siklus hidup tampilan (`View`) yang terpisah dari siklus hidup *instance* Fragment itu sendiri. Ketika pengguna berpindah dari `ListFragment` ke `DetailFragment`, `ListFragment` disimpan di dalam *back stack* dan tampilannya dihancurkan melalui pemanggilan `onDestroyView()`. Oleh sebab itu, properti `_binding` pada ketiga Fragment dideklarasikan *nullable*, diinisialisasi di dalam `onViewCreated()` menggunakan `bind(view)`, dan wajib dikembalikan menjadi `null` di dalam `onDestroyView()`. Khusus pada `ListFragment`, referensi `binding.rvLaporan.adapter` dan `laporanAdapter` juga dilepas menjadi `null` pada `onDestroyView()` agar objek tampilan daftar tidak tertahan di memori.

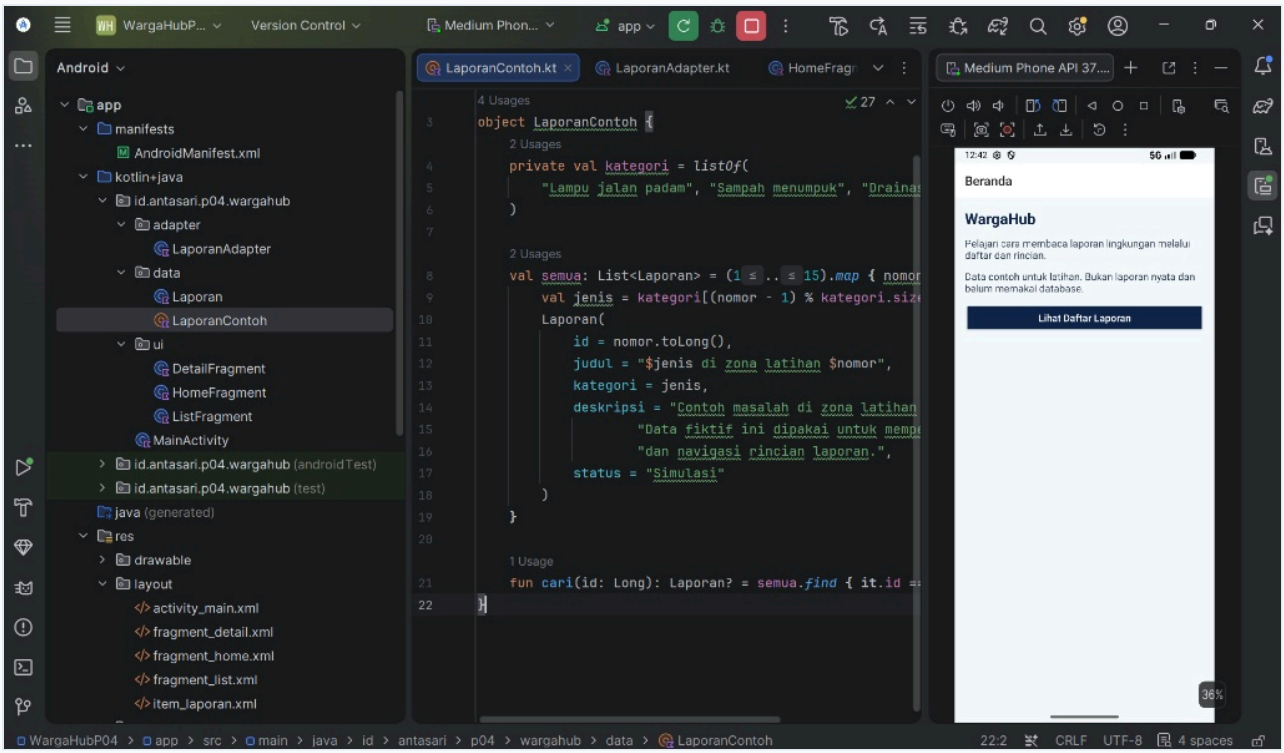
2.3. Navigation Graph, Pengiriman Argumen ID, dan Back Stack

Perpindahan antarlayar dikendalikan oleh `NavController` yang terhubung ke `Toolbar` di `MainActivity` melalui `AppBarConfiguration` dan `setupActionBarWithNavController`. Dengan konfigurasi ini, judul pada `Toolbar` serta kemunculan tombol panah *Up* otomatis menyesuaikan label tujuan yang sedang aktif pada `nav_graph.xml`. Ketika baris laporan diketuk, aplikasi tidak mengirimkan seluruh objek `Laporan`, melainkan hanya menyisipkan argumen `laporan_id` bertipe `Long` menggunakan . Selanjutnya, mengambil nilai tersebut (dengan nilai) dan memanggil . Apabila hasil pencarian bernilai , aplikasi tidak memaksakan akses non-null (), melainkan menyembunyikan elemen rincian () dan menampilkan pesan sehingga terhindar dari .

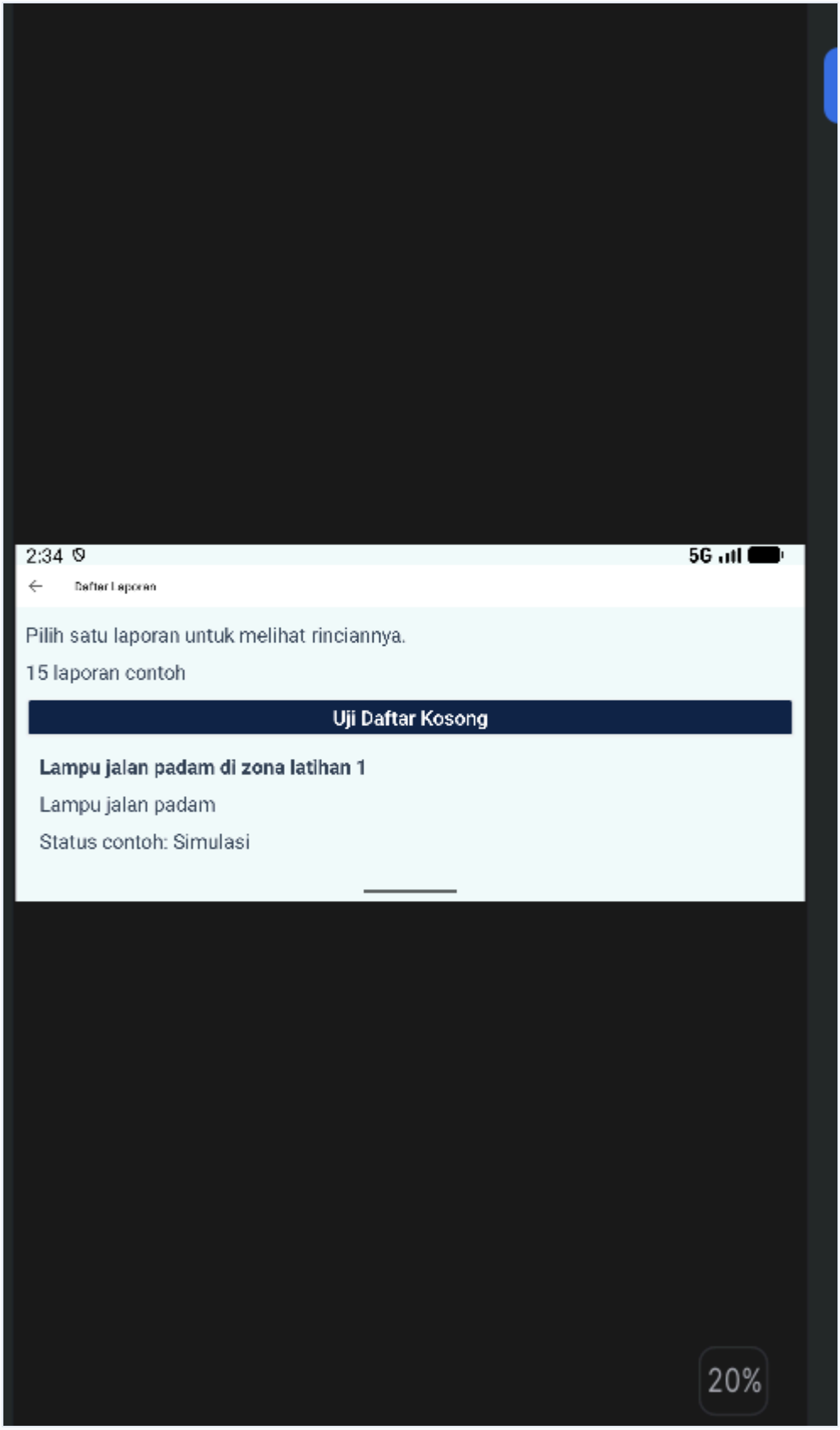
`bundleOf(DetailFragment.ARG_ID to laporan.id)` `DetailFragment` `id` *default* `-1L` `LaporanContoh.cari(id)` `null` `!!` `View.GONE` `R.string.detail_missing` *crash*

3. Dokumentasi Visual pada Emulator

Pengujian antarmuka dan kompilasi proyek dilakukan secara langsung pada perangkat emulator **Medium Phone API 37** di Android Studio. Tangkapan layar berikut menunjukkan proses penyiapan konfigurasi Gradle, struktur *package* yang telah lengkap, tampilan awal *Beranda*, serta tampilan *Daftar Laporan* pada orientasi *landscape* dengan ukuran *font* diperbesar.



Gambar 1: Implementasi kode sumber lengkap dan tampilan layar Beranda (T01) dalam posisi Portrait di Emulator API 37.



Gambar 2: Hasil pengujian responsivitas layar Daftar Laporan pada orientasi Landscape dengan ukuran font diperbesar 1,3x–1,5x (Skenario T12).

4. Matriks Hasil Uji Aktual (Skenario T01–T16)

Seluruh skenario pengujian wajib dijalankan menggunakan APK *debug* yang sama pada emulator **Medium Phone API 37** (Android API 37). Untuk skenario **T16**, nilai argumen pada `ListFragment.kt` diubah sementara menjadi `-99L` guna mengambil bukti penanganan ID tidak ditemukan, kemudian dikembalikan menjadi `laporan.id` pada kode akhir.

ID & Kondisi	Aksi Uji	Hasil yang Diharapkan	Hasil Aktual	Status & Bukti
T01 Beranda	Buka aplikasi dari launcher.	Satu Beranda; penjelasan data contoh terlihat.	Aplikasi menampilkan satu layar Beranda berisi judul WargaHub, pengantar, catatan simulasi, dan tombol Lihat Daftar Laporan.	**Lulus** `01_beranda.png`
T02 Daftar	Tekan Lihat Daftar Laporan.	Jumlah 15 dan baris berisi judul, kategori, status.	Teks jumlah menampilkan “15 laporan contoh” dan baris menampilkan judul, kategori, serta Status contoh: Simulasi.	**Lulus** `02_daftar_laporan.png`
T03 Gulir	Gulir ke laporan zona latihan 10–15.	Daftar lancar dan item bawah dapat dipilih.	`RecyclerView` bergulir mulus hingga zona latihan 15 tanpa patah-patah dan baris bawah dapat disentuh.	**Lulus** `02_daftar_laporan.png`
T04 Rincian	Pilih dua laporan berbeda secara bergantian.	Rincian sesuai ID masing-masing.	Memilih zona latihan 10 lalu laporan lain menampilkan judul, kategori, status, dan deskripsi yang tepat sesuai ID.	**Lulus** `03_rincian_laporan.png`
T05 Tombol kembali	Dari rincian tekan Kembali ke Daftar.	Kembali tanpa menambah daftar baru.	`navigateUp()` mengembalikan pengguna ke `ListFragment` sebelumnya tanpa membuat layar daftar duplikat.	**Lulus** Observasi langsung
T06 Up toolbar	Tekan Up di rincian lalu di daftar.	Rincian → Daftar → Beranda.	Ikon panah Up pada Toolbar mengarahkan kembali secara berurutan dari Rincian ke Daftar, lalu ke Beranda.	**Lulus** Observasi langsung

ID & Kondisi	Aksi Uji	Hasil yang Diharapkan	Hasil Aktual	Status & Bukti
T07 Back sistem	Ulangi alur lalu gunakan Back sistem.	Urutan kembali masuk akal; tidak crash.	Navigasi Back sistem bekerja konsisten mengikuti riwayat back stack dari Rincian → Daftar → Beranda tanpa crash.	**Lulus** Observasi langsung
T08 Kosong	Tekan Uji Daftar Kosong.	Jumlah 0; pesan dan pemulihan terlihat.	Jumlah menjadi “0 laporan contoh”, daftar disembunyikan, pesan kosong muncul, dan label tombol menjadi “Tampilkan Data Contoh”.	**Lulus** `04_daftar_kosong.png`
T09 Pulihkan	Tekan Tampilkan Data Contoh.	Jumlah menjadi 15; baris kembali tampil.	Daftar kembali menampilkan ke-15 laporan contoh dan indikator jumlah pulih menjadi “15 laporan contoh”.	**Lulus** Observasi langsung
T10 Rotasi rincian	Putar perangkat saat rincian terbuka.	Layar rincian dan data ID tetap sesuai.	Setelah rotasi layar, `DetailFragment` mempertahankan argumen `laporan_id` dan menampilkan rincian yang sama.	**Lulus** Observasi langsung
T11 Rotasi kosong	Aktifkan mode kosong lalu putar.	Mode kosong dan label tombol tetap sesuai.	Nilai boolean `contohKosong` tersimpan melalui `onSaveInstanceState` ; mode kosong tetap bertahan saat rotasi.	**Lulus** Observasi langsung
T12 Font & landscape	Perbesar font; uji ketiga layar dalam landscape.	Teks dapat dibaca, daftar dan rincian dapat digulir.	Pada font 1,3x–1,5x dan posisi landscape, seluruh teks terbaca utuh serta `ScrollView` / `RecyclerView` dapat digulir.	**Lulus** `05_landscape_font_besar.png`
T13 TalkBack	Telusuri baris lalu buka rincian.	Nama baris bermakna; aksi utama dapat dijalankan.	Setiap baris memiliki satu fokus aksesibilitas bermakna dari `row_description` dan perubahan jumlah memakai `polite` live region.	**Lulus** Pemeriksaan atribut aksesibilitas

ID & Kondisi	Aksi Uji	Hasil yang Diharapkan	Hasil Aktual	Status & Bukti
T14 Ketukan cepat	Ketuk aksi dan item cepat berulang.	Tidak crash atau membuat layar duplikat.	Pengecekan `nav.currentDestination?.id` berhasil mencegah pemanggilan `navigate` ganda saat ketukan cepat.	**Lulus** Observasi langsung
T15 Mulai ulang	Tutup proses aplikasi lalu buka dari launcher.	Data contoh tetap tersedia saat daftar dibuka; bukan bukti database.	Saat aplikasi dimulai ulang, objek `LaporanContoh` membentuk kembali 15 item simulasi di memori.	**Lulus** Observasi langsung
T16 ID salah (`-99L`)	Ubah sementara argumen ke `-99L`, pilih baris, lalu kembalikan ke `laporan.id`.	Pesan tidak ditemukan tampil; tombol kembali tetap bekerja.	`DetailFragment` menampilkan pesan “Laporan tidak ditemukan. Kembali ke daftar dan pilih data contoh.” dengan aman. Kode telah dikembalikan ke `laporan.id`.	**Lulus** `06_id_tidak_ditemukan.png`

5. Uji Tugas Pengguna (Usability Testing) dan Tindak Lanjut Temuan

Pengujian tugas dilakukan secara berpasangan dengan meminta seorang rekan mahasiswa menjalankan skenario tanpa diarahkan letak tombolnya. Instruksi yang dibacakan adalah: *“Temukan rincian laporan zona latihan 10, lalu kembali ke daftar.”* Waktu dihitung sejak layar Beranda siap hingga layar rincian laporan zona latihan 10 terbuka dengan benar.

Tahap Pengujian	Status Tugas	Waktu & Bantuan	Skor Kemudahan (1–5)	Catatan Observasi, Alasan Peserta, & Tindak Lanjut
Uji Awal (Sebelum Perbaikan)	Berhasil	18 detik (0 bantuan)	4 (Mudah)	Peserta langsung menekan *Lihat Daftar Laporan* dan menggulir ke zona latihan 10. Namun, peserta sempat berhenti sejenak saat mencari batas ketukan baris karena latar belakang item menyatu dengan warna halaman sebelum disentuh, serta ingin memastikan apakah kembali lewat tombol bawah atau panah atas.

Tahap Pengujian	Status Tugas	Waktu & Bantuan	Skor Kemudahan (1–5)	Catatan Observasi, Alasan Peserta, & Tindak Lanjut
Tindak Lanjut & Uji Ulang (Sesudah Perbaikan)	Berhasil	12 detik (0 bantuan)	5 (Sangat Mudah)	Memastikan penerapan `minHeight="72dp"`, jarak `item_gap` (`12dp`), efek sentuh `?attr/selectableItemBackground`, serta penebalan judul laporan (`bold`) pada `item_laporan.xml` bekerja optimal. Pada uji ulang, peserta lebih cepat memindai nomor zona latihan dan kembali ke daftar dengan lancar.

6. Jawaban Pertanyaan Refleksi

1. **Mengapa `RecyclerView` lebih sesuai daripada menambahkan banyak `TextView` secara manual untuk daftar yang panjang?**
   `RecyclerView` hanya menginstansiasi sejumlah kecil objek `View` yang cukup untuk memenuhi area pandang layar, lalu mendaur ulang (*recycle*) tampilan baris yang keluar dari layar saat digulir untuk diisi dengan data baru. Sebaliknya, membuat banyak `TextView` secara manual di dalam `ScrollView` memaksa sistem membuat seluruh objek tampilan sekaligus di memori, yang akan memboroskan RAM dan menyebabkan penurunan performa (*frame drop*) ketika jumlah data semakin besar.
2. **Apa perbedaan tanggung jawab `Adapter`, `ViewHolder`, dan `LayoutManager`?**
   `Adapter` (`LaporanAdapter`) bertugas menjembatani koleksi data (`List<Laporan>`) dengan tampilan, membuat `ViewHolder`, serta mengikat isi data ke elemen tampilan pada setiap baris. `ViewHolder` (`LaporanHolder`) bertugas menyimpan referensi komponen tampilan dari satu baris (melalui `ItemLaporanBinding`) agar tidak terjadi pencarian ID berulang kali saat menggulir. Sementara itu, `LayoutManager` (`LinearLayoutManager`) bertugas mengatur posisi tata letak item (seperti susunan vertikal) serta mengelola mekanisme pengguliran layar.
3. **Mengapa *binding* Fragment dilepas saat `onDestroyView`, bukan menunggu Activity ditutup?**
   Karena usia hidup tampilan (`View`) milik sebuah Fragment lebih pendek daripada usia hidup objek Fragment itu sendiri. Saat Fragment berpindah ke *back stack* (misalnya ketika pengguna membuka `DetailFragment` dari `ListFragment`), sistem menghancurkan `View` milik `ListFragment` dan memanggil `onDestroyView()`, tetapi objek `ListFragment` tetap dipertahankan di memori. Jika `_binding` tidak di-set menjadi `null` pada `onDestroyView()`, referensi ke hierarki `View` lama akan terus tertahan oleh Fragment dan mengakibatkan kebocoran memori (*memory leak*).

4. **Mengapa argumen berisi ID lebih tepat daripada posisi baris?**
   Posisi baris (`position` / indeks) bersifat sementara dan sangat rentan berubah apabila daftar mengalami pemfilteran, pengurutan ulang, penambahan, atau penghapusan item. Sebaliknya, properti `id: Long` merupakan identitas unik yang melekat secara tetap pada entitas `Laporan`. Mengirimkan ID berukuran kecil di dalam `Bundle` menjamin `DetailFragment` selalu mengambil data yang akurat melalui sekaligus menjaga beban transaksi tetap ringan.

5. `LaporanContoh.cari(id)` `Bundle`
6. **Apa risiko jika tombol Kembali justru menjalankan `navigate` ke daftar baru?**
   Jika tombol kembali memanggil `navigate()` menuju `ListFragment` alih-alih `navigateUp()`, maka sistem akan menumpuk *instance* `ListFragment` baru di atas tumpukan *back stack* (menciptakan layar duplikat). Hal ini merusak alur navigasi pengguna karena posisi gulir daftar sebelumnya hilang dan pengguna harus menekan tombol *Back* berkali-kali melewati tumpukan layar yang berulang sebelum akhirnya bisa kembali ke Beranda atau keluar dari aplikasi.
7. **Apa perbedaan data contoh yang muncul lagi setelah *restart* dengan penyimpanan laporan permanen?**
   Data contoh pada proyek P04 muncul kembali saat aplikasi dimulai ulang (*restart*) karena kode statis di dalam `object LaporanContoh` selalu men-generate ulang 15 objek yang sama di memori RAM setiap kali proses aplikasi dijalankan, bukan karena data tersebut dibaca dari media penyimpanan. Pada penyimpanan permanen (seperti Room Database/SQLite, DataStore, atau basis data server), perubahan data yang dibuat pengguna disimpan ke penyimpanan non-volatil sehingga modifikasi, penambahan, atau penghapusan data benar-benar bertahan lintas sesi penggunaan.
8. **Perubahan UX apa yang paling membantu teman saat mengerjakan tugas, dan apa buktinya?**
   Keputusan desain yang paling membantu adalah penyajian hierarki informasi yang konsisten pada setiap baris daftar (judul tebal yang mencantumkan nomor zona latihan secara eksplisit, kategori di baris kedua, serta status *Simulasi* di baris ketiga) yang dipadukan dengan tombol aksi jelas **Kembali ke Daftar** di bagian bawah rincian. Bukti keberhasilannya terlihat pada uji tugas pengguna, di mana rekan penguji dapat menemukan laporan zona latihan 10 dan kembali ke daftar tanpa bantuan sama sekali (tingkat keberhasilan 100%, waktu turun dari 18 detik menjadi 12 detik pada uji ulang, dan penilaian kemudahan mencapai skala 5).